

8INF876 – Conception et architecture des
systèmes d'infonuagique

Etude et conception d'une Ar- chitecture IoT avec MQTT

EL NAGGAR Sofiane
HANY Matéo
TONNERRE Aubry

May 2, 2025

Table des matières

Table des matières	1
Liste de source codes	2
Liste des accronymes	3
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Description de la problématique	1
1.3 Description du développement	2
2 APPROCHE PRISE POUR LA CONCEPTION	4
2.1 Introduction	4
2.2 Architecture du système	4
2.3 Diagrammes de cas d'utilisations	5
2.4 Diagrammes de séquences	6
2.5 Diagrammes d'activités	7
2.5.1 Inscription d'un utilisateur	7
2.5.2 Connexion d'un utilisateur	8
2.5.3 Accéder aux données en temps réel	9
2.5.4 Gestion des abonnements	10
2.5.5 Publication des données par les capteurs	11
2.5.6 Consultation de l'historique des données	12
2.6 Conclusion	13
3 IMPLEMENTATION ET RÉSULTATS	14
3.1 Introduction	14
3.2 Outils d'implémentation	14
3.3 Approche prise pour l'implémentation	15
3.3.1 ESP32	15
3.3.2 MQTT Broker	15
3.3.3 MQTT Listener	16
3.3.4 Mysql DB	16
3.3.5 API rest	17
3.3.6 Websocket	18
3.3.7 Front	18
3.3.8 Orchestration Kubernetes	20
3.4 Conclusion	21
4 Conclusion	22
Bibliographie	23

Liste des Figures

1	Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025 (in billions)[5]	1
2	Architecture du Système	4
3	Diagramme des cas d'utilisation	5
4	Diagramme séquence	6
5	Diagramme d'activités : Inscription d'un utilisateur	7
6	Diagramme d'activités : Connexion d'un utilisateur	8
7	Diagramme d'activités : Accéder aux données en temps réel	9
8	Diagramme d'activités : Gestion des abonnements	10
9	Diagramme d'activités : Publication des données par les capteurs	11
10	Diagramme d'activités : Consultation de l'historique des données	12
11	Exemple montage ESP32	16
12	Exemple Message MQTT ESP32 non formaté	16
13	Base de donnée MySQL	17
14	Page login	19
15	Page abonnement - Topics non sélectionné	19
16	Page abonnement - Topics sélectionné	19
17	Page accueil	20

Glossaire

IoT - Internet des Objets . 1

MQTT - Message Queing Telemetry Transport . 1

1 INTRODUCTION

1.1 Introduction

L'Internet des objets (IoT) facilite notre quotidien en intégrant des dispositifs connectés capables de surveiller, automatiser et optimiser différents aspects de la vie. Selon une étude de IHS datant de 2023 (Figure 1), En 2015, environ 15 milliards de dispositifs IoT étaient utilisés dans le monde. Ce nombre pourrait atteindre 75 milliards d'ici 2025, ce qui montre une croissance exponentielle.

Cette évolution ouvre des opportunités importantes dans des domaines comme la maison intelligente, la gestion des ressources et la sécurité.

La clé de cette facilité réside dans la communication entre les dispositifs connectés. Les technologies sont diverses mais elles se basent surtout sur des protocoles spécifiques conçus pour les environnements contraints en termes de ressources, comme le Message Queuing Telemetry Transport (MQTT)[8]. Ce protocole, léger et efficace, est particulièrement adapté aux architectures IoT grâce à sa capacité à transmettre des messages sur des réseaux instables ou à faible bande passante. L'utilisation de MQTT contribue à l'optimisation des échanges de données, en garantissant une latence minimale et une grande fiabilité, des éléments essentiels dans des applications critiques telles que les soins de santé connectés ou la surveillance industrielle.

L'intérêt croissant pour MQTT découle également de sa simplicité d'implémentation et de son modèle pub/sub (publication/abonnement), qui facilite la gestion de nombreuses connexions simultanées. À mesure que les réseaux IoT deviennent de plus en plus grands et complexes, il est important d'utiliser des protocoles évolutifs capables de répondre aux besoins croissants des utilisateurs finaux tout en minimisant l'impact sur les ressources des systèmes embarqués.

Dans le cadre de ce projet, une architecture IoT basée sur MQTT a été développée et testée. L'objectif principal était de démontrer la robustesse de cette solution dans un scénario réel. Ce travail répond à la problématique suivante : Comment utiliser le protocole MQTT pour concevoir une architecture IoT scalable, performante et résiliente face aux contraintes des environnements connectés ?

1.2 Description de la problématique

Malgré les avantages prometteurs des architectures IoT basées sur MQTT, leur mise en oeuvre soulève plusieurs défis techniques et opérationnels. D'une part, la scalabilité du système peut être limitée lorsqu'un grand nombre de dispositifs doivent communiquer simultanément, entraînant des problèmes de surcharge et une dégradation des performances réseau. D'autre part, la résilience face aux pertes de connexion, fréquentes dans les réseaux IoT, demeure un enjeu critique pour garantir l'intégrité et la continuité des flux de données. Dans ce projet, nous avons utilisé **Kubernetes** (avec *K3S*) pour orchestrer les services liés à notre architecture MQTT. Cette solution permet d'assurer

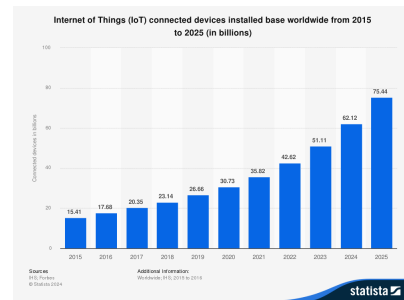


Figure 1: Internet of Things (IoT) connected devices installed base worldwide from 2015 to 2025 (in billions)[5]

une gestion optimisée des ressources, de faciliter le déploiement et de renforcer la capacité de l'architecture à évoluer dynamiquement. Plus précisément, l'utilisation de Kubernetes a permis de répondre aux problèmes suivants :

- **Optimisation des performances réseau** pour gérer efficacement les flux de données provenant d'un grand nombre de dispositifs connectés.
- **Amélioration de la résilience** grâce à des mécanismes robustes de gestion des erreurs et de reprise après déconnexion.
- **Simplification du déploiement et de la gestion** grâce à l'automatisation des opérations et à la scalabilité dynamique des conteneurs.

Cette approche vise à démontrer que l'association de **MQTT et Kubernetes**, lorsqu'elle est correctement implémentée, constitue une solution robuste et évolutive pour relever les défis actuels des architectures IoT à grande échelle [12].

1.3 Description du développement

Plusieurs composantes ont été développées pour assurer le bon fonctionnement de l'ensemble de l'architecture, couvrant à la fois les dispositifs matériels, le serveur MQTT et l'interface client.

Dans un premier temps, du code a été développé pour des microcontrôleurs **ESP32**, dans le but de collecter des données via des capteurs et les transmettent au serveur MQTT à l'aide du bluetooth. Ces dispositifs communiquent avec le broker MQTT en utilisant un mécanisme de publication des messages vers des *topics* spécifiques, garantissant une transmission en temps réel des informations.

Ensuite, un **MQTT Listener** a été implémenté pour surveiller les événements publiés sur le broker MQTT. Ce composant agit comme un pont entre le serveur MQTT et la base de données. Il reçoit les messages des différents *topics* et les stocke dans une base de données centralisée, permettant ainsi une conservation fiable et organisée des données pour des analyses ultérieures.

Par ailleurs, un **front-end avec API** a été développé pour offrir une interface utilisateur simple et accessible. Cette interface permet aux clients de s'abonner aux *topics* gérés par le broker MQTT, leur donnant ainsi la possibilité de recevoir en direct les données les plus récentes et de suivre les événements d'intérêt. Le front-end s'intègre de manière transparente avec l'architecture backend pour garantir une expérience utilisateur fluide.

La méthodologie adoptée repose sur une approche itérative, combinant développement, tests fonctionnels. L'objectif principal était de garantir la robustesse et la scalabilité du système tout en minimisant les temps de latence dans la communication entre les composants.

Les résultats attendus incluent :

- Une transmission fiable des données collectées par les ESP32 vers le broker MQTT.
- Une gestion efficace des messages publiés, avec leur sauvegarde sécurisée dans la base de données.

- Une interface utilisateur permettant une interaction dynamique avec le système via des abonnements aux *topics*.

Ce développement pose ainsi les bases de notre architecture IoT pour un objectif de performante et de résilience pour démontrer la pertinence de l'utilisation du protocole MQTT dans des environnements connectés.

2 APPROCHE PRISE POUR LA CONCEPTION

2.1 Introduction

Ce chapitre présente la conception de l'architecture de protocole de communication MQTT, conçue pour garantir un fonctionnement fiable et efficace. L'objectif est d'offrir une compréhension claire des composants du système, de leur organisation, et de leurs interactions.

Les sections suivantes permettront de :

- Détailler l'architecture générale du système avec un schéma illustrant les interactions entre les différents composants, tels que les échanges client/serveur, les appels API, et la gestion des données.
- Mettre en lumière les principales fonctionnalités du système à travers des **diagrammes de cas d'utilisation**, décrivant les interactions entre les utilisateurs et les différents éléments.
- Illustrer les flux de données en temps réel grâce aux **diagrammes de séquences**, qui montrent les échanges entre les capteurs, le broker MQTT, le Raspberry Pi, et le frontend.
- Décrire les étapes clés du fonctionnement à l'aide de **diagrammes d'activités**, qui synthétisent les processus du système.

2.2 Architecture du système

Voici le diagramme de l'architecture MQTT :

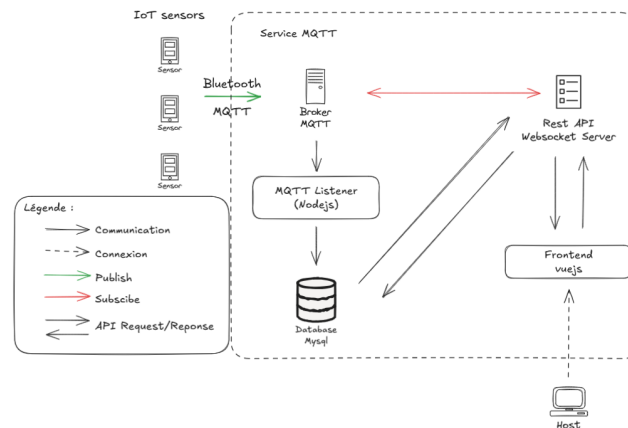


Figure 2: Architecture du Système

Le système de communication basé sur le protocole MQTT repose sur une architecture modulaire, conçue pour assurer une gestion efficace et en temps réel des données IoT. Les capteurs collectent les données et les publient via Bluetooth vers le broker MQTT, où elles sont diffusées sur des *topics* spécifiques.

Un **MQTT Listener**, développé en `Node.js`, écoute les événements du broker et enregistre les messages dans une base de données `MySQL`, assurant ainsi la persistance des données. Une

API REST permet au frontend d'accéder à ces informations en offrant des fonctionnalités pour récupérer l'historique des données.

En parallèle, un **serveur WebSocket** récupère les messages en direct du broker MQTT et les redirige directement aux clients abonnés, leur permettant de recevoir les mises à jour en temps réel. Le **frontend** offre une interface utilisateur permettant de s'abonner aux *topics* et de consulter l'historique des données.

2.3 Diagrammes de cas d'utilisations

Avant de déployer une architecture et de développer une interface utilisateur, il est essentiel de définir les cas d'utilisation associés à l'outil.

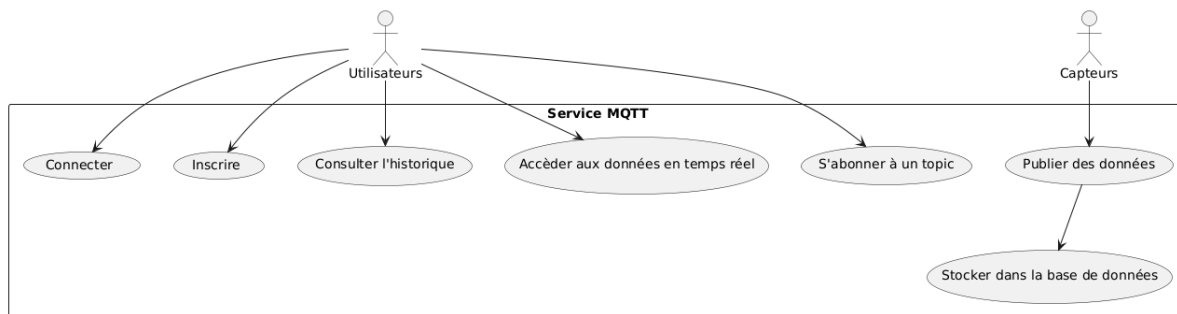


Figure 3: Diagramme des cas d'utilisation

Dans un premier temps, les dispositifs connectés doivent être en mesure de publier des informations via le broker MQTT. L'architecture mise en place doit permettre non seulement la publication des données, mais aussi leur enregistrement dans une base de données afin de fournir un historique complet des mesures collectées.

Par ailleurs, l'application utilisateur offre des fonctionnalités comme la possibilité de pouvoir s'inscrire et se connecter à l'interface pour accéder à différents services. Parmi ces services figurent la consultation de l'historique des données stockées en base de données ainsi que l'accès en temps réel aux mesures diffusées par le broker MQTT. L'abonnement à des *topics* spécifiques garantit une personnalisation de l'expérience utilisateur en fonction des besoins individuels.

2.4 Diagrammes de séquences

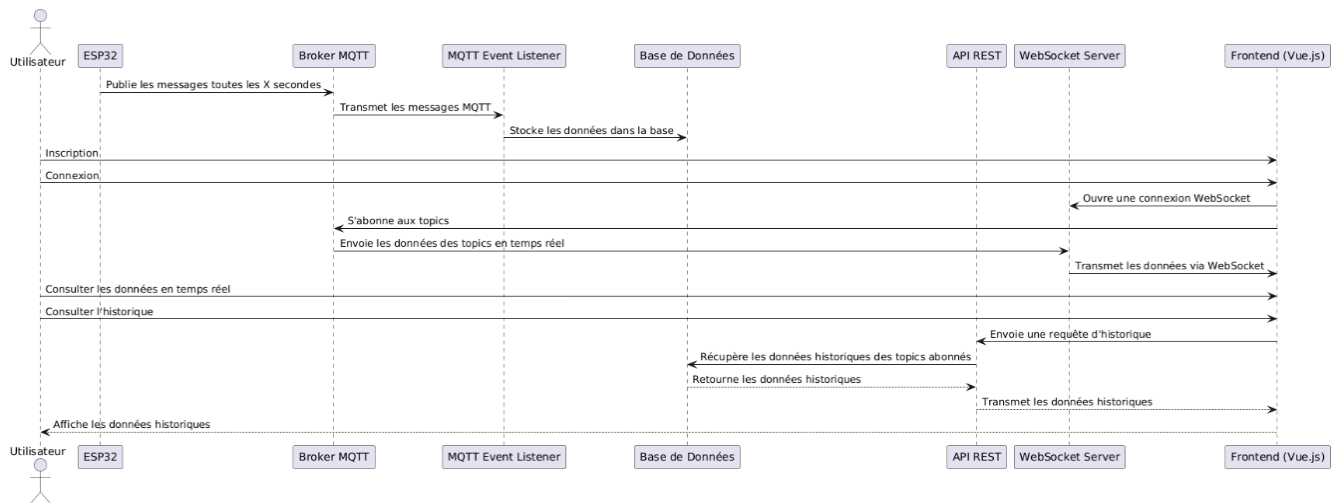


Figure 4: Diagramme séquence

Ce diagramme illustre le fonctionnement global du système IoT basé sur le protocole MQTT, en détaillant les interactions entre les différents composants et les étapes principales.

Les capteurs ESP32, connectés aux différents dispositifs, publient des messages toutes les X secondes via le protocole MQTT. Ces messages contiennent les données collectées, telles que la détection de mouvement, les relevés de température ou la luminosité. Ces données sont ensuite transmises au broker MQTT, qui agit comme un intermédiaire pour gérer les publications et les abonnements.

Le MQTT Event Listener écoute en continu les messages publiés sur le broker MQTT. Ces messages sont ensuite stockés dans une base de données pour assurer leur persistance et permettre leur utilisation ultérieure. Lorsqu'un utilisateur souhaite accéder au système, il peut s'inscrire ou se connecter via le frontend, qui représente l'interface utilisateur. L'API REST gère les processus d'inscription et d'authentification en enregistrant les informations dans la base de données.

Une fois connecté, l'utilisateur peut sélectionner les topics auxquels il souhaite s'abonner, comme les alertes de mouvement ou les relevés de température. Ces abonnements sont enregistrés dans la base de données via l'API REST. Les données des topics abonnés sont ensuite envoyées en temps réel par le broker MQTT au frontend, en passant par le WebSocket Server. Cela permet à l'utilisateur de visualiser les informations en continu sur l'interface utilisateur.

Si l'utilisateur souhaite consulter l'historique des données, une requête est envoyée depuis le frontend à l'API REST. Cette dernière interroge la base de données pour récupérer les données historiques des topics auxquels l'utilisateur est abonné. Les données sont ensuite renvoyées au frontend pour affichage. Cette page ouvre une connexion **Websocket** pour permettre à l'utilisateur de recevoir les données des capteurs ou il est abonné en continue.

2.5 Diagrammes d'activités

2.5.1 Inscription d'un utilisateur

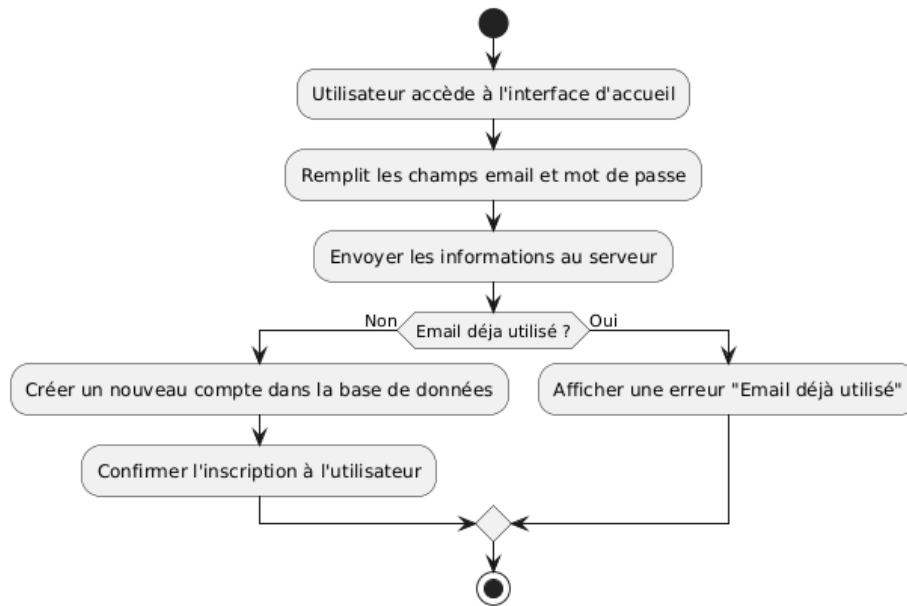


Figure 5: Diagramme d'activités : Inscription d'un utilisateur

Ce diagramme d'activités illustre le processus d'inscription d'un utilisateur au système. L'utilisateur commence par accéder à l'interface d'inscription, où il remplit les champs requis, comme son adresse email et son mot de passe, puis envoie ces informations au serveur.

Le serveur reçoit les informations et vérifie si l'adresse email existe déjà dans la base de données. Si l'email est déjà utilisé, un message d'erreur "Email déjà utilisé" est affiché, et l'utilisateur est redirigé directement vers l'étape où il doit à nouveau remplir les champs email et mot de passe.

En revanche, si l'email n'existe pas, un nouveau compte est créé avec les informations fournies, et une confirmation d'inscription est envoyée pour informer l'utilisateur que son compte a été créé avec succès. Ce processus garantit une gestion claire et sécurisée des nouvelles inscriptions tout en évitant les doublons dans la base de données.

2.5.2 Connexion d'un utilisateur

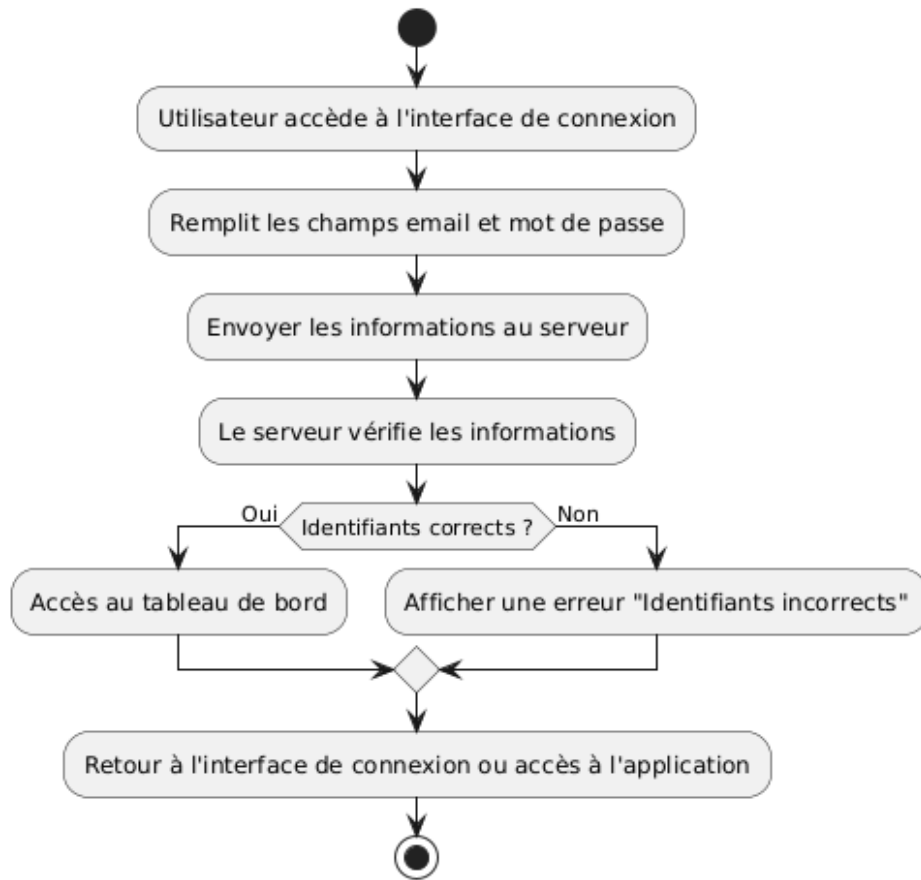


Figure 6: Diagramme d'activités : Connexion d'un utilisateur

Ce diagramme d'activités décrit le processus de connexion d'un utilisateur au système. L'utilisateur commence par accéder à l'interface de connexion, où il saisit son adresse email et son mot de passe dans les champs prévus à cet effet. Une fois ces informations remplies, il les envoie au serveur pour vérification.

Le serveur reçoit les informations et vérifie si les identifiants sont corrects, c'est-à-dire si l'adresse email et le mot de passe correspondent à un compte existant dans la base de données. Si les identifiants sont corrects, l'utilisateur obtient un accès à l'interface des abonnements, où il peut gérer ses topics ou consulter les données disponibles.

En revanche, si les identifiants sont incorrects, un message d'erreur "Identifiants incorrects" est affiché à l'utilisateur, qui est automatiquement redirigé vers l'étape où il doit à nouveau remplir les champs email et mot de passe. Ce mécanisme permet à l'utilisateur de réessayer sans quitter le processus de connexion, tout en garantissant une vérification sécurisée des accès pour s'assurer que seuls les utilisateurs autorisés peuvent accéder au système.

2.5.3 Accéder aux données en temps réel

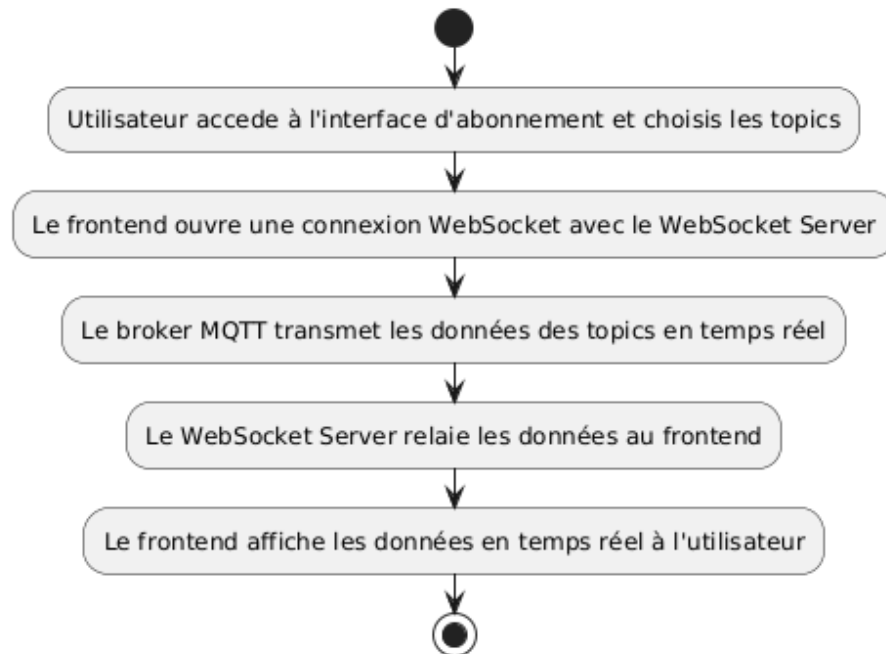


Figure 7: Diagramme d'activités : Accéder aux données en temps réel

Ce diagramme d'activités illustre le processus de consultation des données en temps réel par un utilisateur. Tout commence lorsque l'utilisateur accède à l'interface d'abonnement et sélectionne les topics auxquels il souhaite s'abonner, comme des alertes ou des données spécifiques des capteurs.

Une fois les topics sélectionnés, le frontend établit une connexion avec le WebSocket Server. Ce dernier est responsable de la transmission des données en temps réel. Le broker MQTT, qui centralise les données publiées par les capteurs, transmet ensuite les informations des topics sélectionnés au WebSocket Server.

Le WebSocket Server redirige ces données directement au frontend, qui les affiche en temps réel à l'utilisateur sous une forme claire et intuitive. Ce processus garantit une mise à jour continue et immédiate des informations, permettant à l'utilisateur de suivre l'évolution des données en temps réel.

2.5.4 Gestion des abonnements

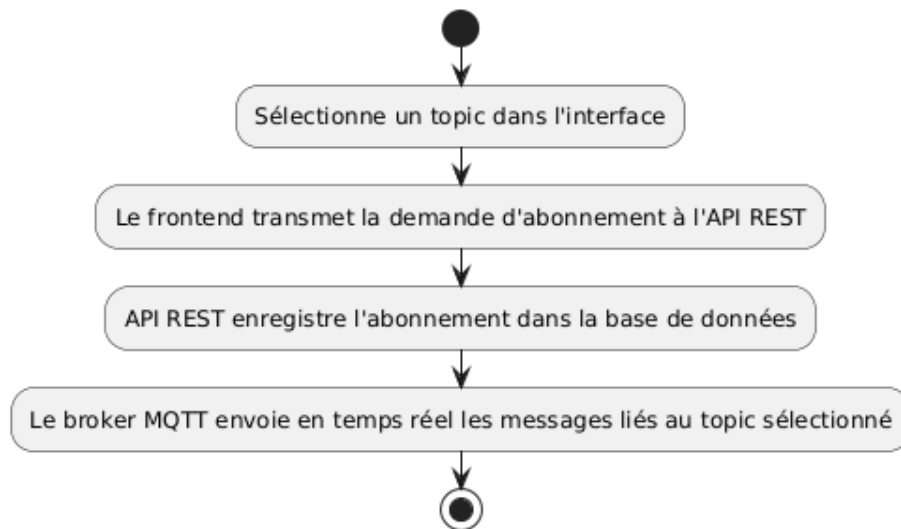


Figure 8: Diagramme d'activités : Gestion des abonnements

Ce diagramme d'activités représente le processus d'abonnement d'un utilisateur à un topic spécifique. L'utilisateur commence par sélectionner un topic dans l'interface, correspondant à une catégorie de données ou de notifications qu'il souhaite recevoir, comme les relevés de température ou les alertes de mouvement.

Une fois le topic choisi, le frontend transmet la demande d'abonnement à l'API REST. L'API REST se charge alors d'enregistrer cet abonnement dans la base de données, assurant une persistance des préférences de l'utilisateur.

Enfin, après l'enregistrement, le broker MQTT envoie en temps réel les messages liés au topic sélectionné, permettant à l'utilisateur de recevoir et de suivre les données associées directement depuis l'interface. Ce processus garantit une gestion fluide et dynamique des abonnements tout en offrant une expérience utilisateur optimisée.

2.5.5 Publication des données par les capteurs

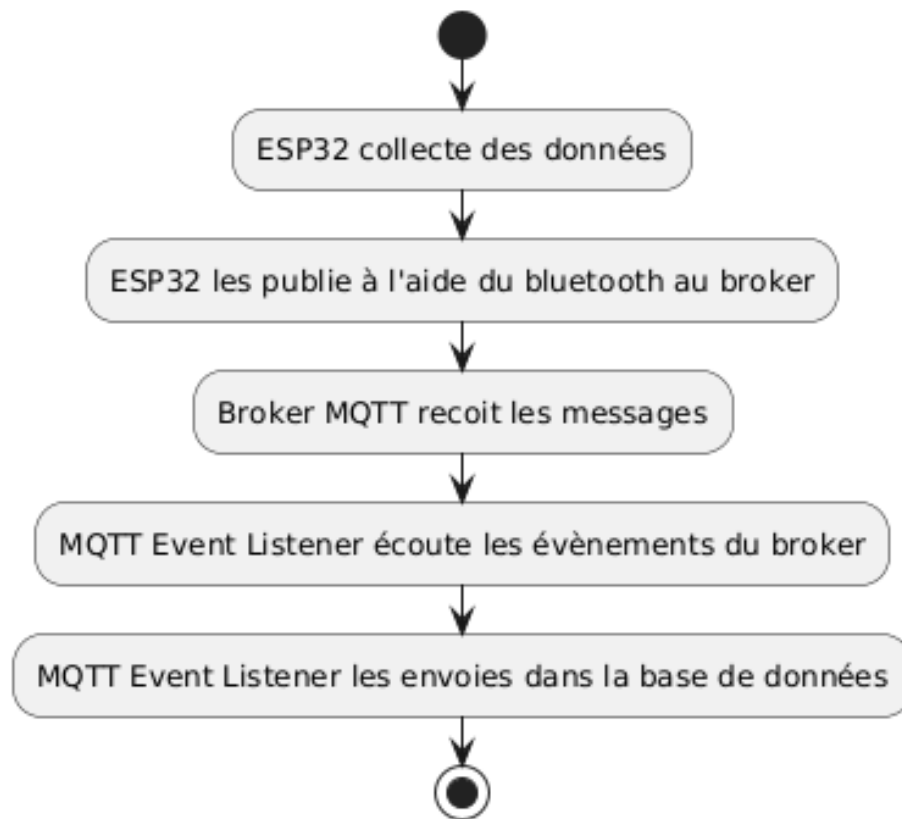


Figure 9: Diagramme d'activités : Publication des données par les capteurs

Ce diagramme d'activités illustre le processus de publication et de traitement des données collectées par les capteurs ESP32. Tout commence avec les ESP32 qui collectent des données, telles que des relevés de température, des niveaux de luminosité ou des détections de mouvement.

Les ESP32 publient ensuite ces données via Bluetooth au broker MQTT. Une fois les messages reçus par le broker, celui-ci les met à disposition pour être consommés par d'autres composants du système.

Le MQTT Event Listener joue un rôle clé dans ce processus : il écoute en continu les événements et messages provenant du broker MQTT. Les messages collectés sont ensuite envoyés à la base de données par le MQTT Event Listener, garantissant la persistance des informations collectées pour un usage ultérieur. Ce processus assure une transmission fluide des données, depuis les capteurs jusqu'à leur stockage sécurisé.

2.5.6 Consultation de l'historique des données

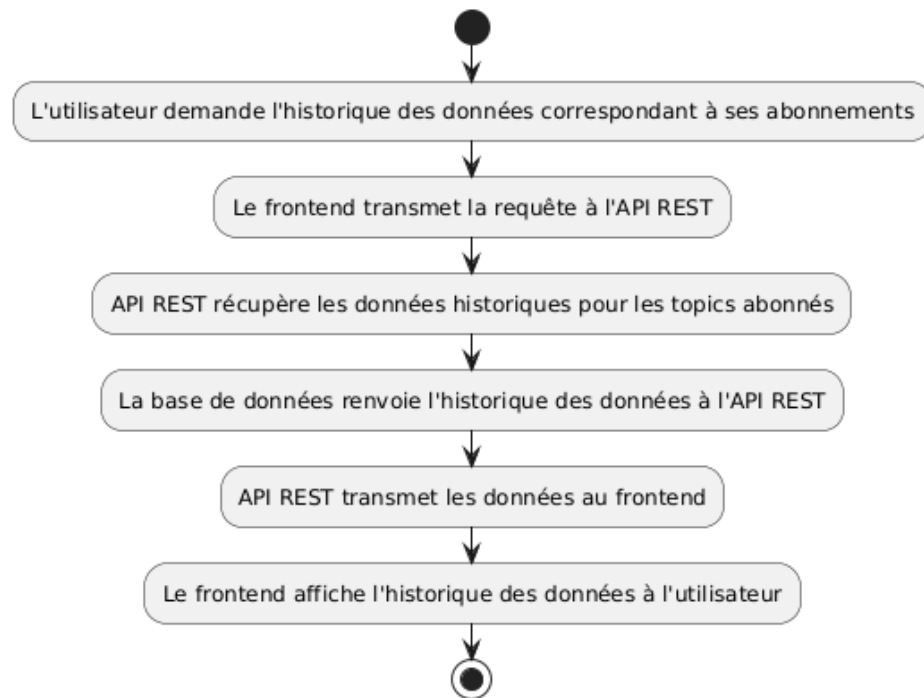


Figure 10: Diagramme d'activités : Consultation de l'historique des données

Ce diagramme d'activités représente le processus par lequel un utilisateur consulte l'historique des données liées à ses abonnements. L'utilisateur commence par demander, via l'interface, l'historique des données correspondant aux topics auxquels il s'est abonné.

Le frontend transmet cette requête à l'API REST, qui se charge de récupérer les données historiques pour les topics abonnés. Pour ce faire, l'API REST interroge la base de données, où sont stockées les données précédemment collectées et persistantes.

Une fois les données récupérées, la base de données renvoie l'historique à l'API REST. L'API REST transmet ensuite ces données au frontend. Enfin, le frontend affiche l'historique des données de manière claire et lisible pour l'utilisateur, lui permettant de visualiser les informations nécessaires à partir de ses abonnements. Ce processus garantit une gestion fluide et précise des données historiques tout en offrant une expérience utilisateur optimisée.

2.6 Conclusion

Ce chapitre a présenté la manière dont notre architecture IoT basé sur MQTT a été conçue et structurée, en expliquant les principaux composants et leurs interactions. L'architecture intègre des capteurs connectés aux ESP32, un Raspberry Pi orchestré avec K3S, un broker MQTT Mosquitto, une base de données centralisée, une API REST et une interface utilisateur. Elle a été pensée pour être modulaire, évolutive et fiable, afin de répondre aux besoins d'un système IoT performant.

Les diagrammes utilisés ont permis d'illustrer les aspects fonctionnels du système. Les diagrammes de cas d'utilisation ont montré comment les utilisateurs interagissent avec les différents éléments, tandis que les diagrammes de séquences et d'activités ont détaillé le fonctionnement des échanges de données et des processus internes. Ces représentations ont aidé à mieux comprendre l'architecture et la manière dont les différents composants travaillent ensemble.

3 IMPLEMENTATION ET RÉSULTATS

3.1 Introduction

Ce chapitre est consacré à la mise en œuvre de l'architecture IoT et des fonctionnalités associées, ainsi qu'à la validation de leur bon fonctionnement par des tests pratiques. L'objectif principal est de présenter les outils utilisés pour l'implémentation, de décrire les différentes étapes de développement et de démontrer les résultats obtenus.

La première partie décrit les outils d'implémentation, incluant les environnements matériels et logiciels sélectionnés pour répondre aux exigences du projet. Ensuite, les différentes composantes de l'architecture, telles que les microcontrôleurs ESP32, le broker MQTT, le MQTT Listener, la base de données MySQL, le serveur WebSocket, l'API REST et le frontend, sont détaillées dans des sous-sections spécifiques. Chacune de ces parties met en avant les étapes d'implémentation, illustrées par des extraits de code et des captures d'écran.

Enfin, les résultats des tests fonctionnelles sont discutés pour évaluer les fonctionnalités développées et identifier les éventuelles limitations. Cette analyse permet de valider la conformité du système avec les objectifs initiaux et de tirer des conclusions sur les performances globales de l'architecture.

3.2 Outils d'implémentation

L'implémentation du projet a nécessité l'utilisation d'une combinaison d'outils matériels et logiciels pour répondre aux exigences fonctionnelles et garantir la robustesse de l'architecture. Les outils utilisés sont les suivants :

- **Matériel :**
 - **ESP32:** Microcontrôleur utilisé pour collecter et transmettre des données via le protocole MQTT.
 - **Raspberry Pi 4 :** Dispositif utilisé comme serveur pour héberger le broker MQTT et d'autres services nécessaires au projet.
- **Environnement de développement :**
 - **Arduino IDE [13]:** Utilisé pour programmer les ESP32.
 - **WebStorm [16] et Visual Studio Code [14]:** IDEs employés pour le développement des composants backend et frontend.
- **Frameworks et bibliothèques :**
 - **PubSubClient [11] :** Bibliothèque pour gérer la communication MQTT sur les microcontrôleurs ESP32.
 - **Express.js [3]:** Framework backend utilisé pour créer l'API REST permettant l'interaction avec la base de données.

- **Vue.js** [15]: Framework frontend utilisé pour concevoir l’interface utilisateur.
- **Bases de données :**
 - **MySQL** [9]: Base de données utilisée pour la persistance des données collectées.
 - **DBeaver-CE** [1] : Outil graphique pour gérer et interagir avec la base de données MySQL.
- **Communication et orchestration :**
 - **Eclipse Mosquitto** [2]: Broker MQTT léger et performant utilisé pour la communication entre les différents composants.
 - **MQTTX** [4]: Client MQTT utilisé pour tester les publications et abonnements sur le broker.
 - **K3S** [6] : Distribution allégée de Kubernetes utilisée pour orchestrer les services déployés sur le Raspberry Pi.
- **Langages de programmation :**
 - **Node.js** [10]: Utilisé pour le développement du MQTT Listener et du backend de l’API REST.
 - **JavaScript** [7]: Principal langage utilisé pour le frontend et le backend.

3.3 Approche prise pour l’implémentation

L’ensemble des programme son disponible dans le ZIP joint au rapport .

3.3.1 ESP32

Deux ESP32 ont été programmés en utilisant la bibliothèque **PubSubClient** pour gérer la communication via le protocole MQTT. Les données sont transmises au système en utilisant le WiFi, où la bibliothèque convertit automatiquement les messages au format JSON compatible MQTT. Les ESP32 collectent au total trois types de données : luminosité, humidité et état de la lumière pour notre premier cas d’usage. Les mesures sont envoyées au broker MQTT toutes les 30 secondes, assurant une transmission régulière et fiable.

Voici un exemple de montage :

3.3.2 MQTT Broker

Pour le déploiement du broker MQTT, une image Docker développée par **Eclipse Mosquitto** a été utilisée. Cette image offre une solution légère et facilement déployable pour mettre en place l’infrastructure MQTT. L’utilisation de conteneurs Docker a permis de simplifier la gestion du broker en encapsulant toutes les dépendances nécessaires au sein de l’image.

Afin d’orchestrer et de gérer efficacement cette image Docker, une solution Kubernetes a été mise en place. Kubernetes a été utilisé pour automatiser le déploiement, la gestion des conteneurs,

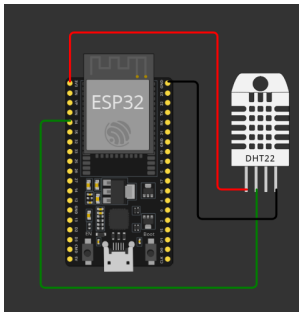
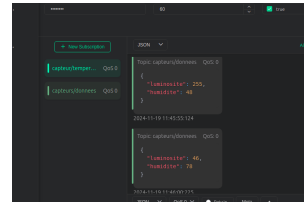


Figure 11: Exemple montage ESP32

Figure 12: Exemple Message MQTT ESP32 **non formaté**

et assurer une scalabilité dynamique. Dans cette configuration, il est nécessaire de paramétrer les données spécifiques au conteneur, notamment les ports de communication. Par défaut, le port 1883 a été utilisé pour les communications standard MQTT. Toutefois, l'architecture est également compatible avec l'utilisation du port 8883 pour les communications sécurisées via TLS/SSL, bien que cette dernière option n'ait pas été implémentée dans la version actuelle du système.

L'image Docker intègre une fonctionnalité de stockage interne des données de communication. Cependant, cette option n'a pas pu être utilisée dans le cadre de ce projet. Il a donc été décidé de déployer une solution supplémentaire au sein de l'infrastructure pour gérer ce besoin.

3.3.3 MQTT Listener

Le `mqtt-listener` est un service dédié à l'écoute des messages publiés sur un broker MQTT et à les stockés dans une base de données MySQL. Sa configuration repose sur des variables d'environnement ou des valeurs par défaut pour établir les connexions au broker et à la base de données. Lors de son exécution, le service vérifie la présence des tables nécessaires (`users`, `topics`, `messages`, `subscriptions`) et les crée si elles n'existent pas. Une fois connecté au broker, le client MQTT s'abonne à tous les topics (`#`). À chaque réception d'un message, le service convertit le contenu en chaîne de caractères, vérifie l'existence du topic dans la base de données (et l'ajoute si nécessaire), puis stocke le message dans la table `messages` avec une référence au topic correspondant. Une gestion d'erreurs est mise en place pour traiter les problèmes liés aux connexions ou au traitement des données. Ce processus est initialisé par une fonction principale (`start`) qui configure les connexions et les abonnements nécessaires.

3.3.4 Mysql DB

Une base de données MySQL est mise en place et repose sur une structure relationnelle conçue pour gérer efficacement les interactions entre les utilisateurs, les sujets de discussion (topics) et les messages échangés. Elle se compose de quatre entités principales : `users`, `topics`, `messages`, et `subscriptions`.

L'entité `users` contient les informations nécessaires à l'identification et à l'authentification des utilisateurs, notamment leur adresse email et leur mot de passe hashé, chaque utilisateur étant identifié de manière unique par un `id`. L'entité `topics` représente les différents sujets auxquels les utilisateurs peuvent s'abonner, chaque topic étant défini par un `id` unique et un nom descriptif. Les messages échangés sont stockés dans l'entité `messages`, qui associe chaque message à un topic

via la clé étrangère `topic_id`, tout en enregistrant le contenu du message et son heure d'envoi (`timestamp`).

Enfin, la table `subscriptions` joue un rôle de pivot, permettant de modéliser la relation N:M entre les utilisateurs et les topics. Elle enregistre les abonnements des utilisateurs en associant les clés étrangères `user_id` et `topic_id`. Cette organisation permet de maintenir une traçabilité complète des interactions utilisateur tout en garantissant une extensibilité et une maintenabilité optimales de la base de données.

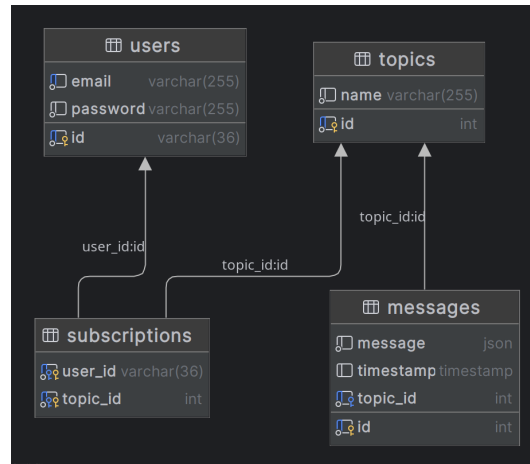


Figure 13: Base de donnée MySQL

3.3.5 API rest

En plus de la websocket, nous avons donc mis en place une API Rest, servant uniquement à communiquer entre le front et la base de données. Également développée en Node.js, elle utilise le framework ExpressJS comme expliqué précédemment. Résumons ses multiples actions et connexion :

- **/users/signup** : Pour commencer, la première action de l'API Rest est d'inscrire un nouveau client. En lien avec la page de login, cette dernière vérifie si un mail et un mot de passe ont bien été entré par l'utilisateur. Elle vérifie ensuite si le mail n'est pas déjà utilisé. Si tout est valide, un message de confirmation (200) est envoyé au client.
- **/users/login** : De même que pour l'inscription, l'API se charge évidemment des connexions des clients. Cette fois, nous vérifions si le couple mail/mot de passe est valide et renvoyons la confirmation ou non au client.
- **/topics** : En lien avec la page d'abonnement, cette route a pour objectif de récupérer la liste des topics présent dans la base de données et les retransmettre au client. En effet, le nombre de topics étant dynamique dans notre architecture (l'utilisateur peut décider d'ajouter ou d'enlever des capteurs.), il est indispensable de fournir au démarrage une liste à jour.
- **/subscriptions/current** : Toujours en lien avec la page d'abonnement, l'objectif ici est de récupérer les abonnements actuels de l'utilisateur pour les afficher correctement au client.

- **/topics/subscribe** : À l'inverse de la route précédente, ici l'objectif est de mettre à jour les abonnements de l'utilisateur dans le cas où ce dernier les aurait modifiés.
- **/topics/data** : Seul route en lien avec la page d'accueil, elle est probablement l'une des plus importante. En effet, son rôle est de transmettre au client les données des topics qu'il souhaite. Les données récupérées ici ne sont pas en temps réel, la websocket se charge de cela, seul l'historique des mesures présentes dans la base de données sont transmises.

Pour conclure, l'API Rest a pour objectif de connecter le client et la base de données du broker. Elle est majoritairement utilisée pour les connexions clientes et l'initialisation des pages.

3.3.6 Websocket

Afin de pouvoir fournir au client les données envoyées par les abonnements du broker en temps réel, nous avons mis en place une websocket. Bien que nous ayons dans un premier temps mis en place une API Rest pour la communication entre le client et le broker, nous avons rapidement constaté que de l'envoi de donnée en continu du serveur vers le client ne pouvait pas se faire avec cette technologie.

La websocket est donc entièrement développée en Node.js et utilise la librairie WebSocket. On y retrouve la majorité des communications entre le client et le broker faite à l'aide de la librairie mqtt.

Nous avons donc choisi d'implémenter, en plus de l'API Rest, une websocket. Son fonctionnement a été réduit à strict minimum. En effet, lors d'une connexion d'un client à cette dernière, la socket associe l'id de base de données du client à son id de session afin de pouvoir redistribuer correctement les données des différents abonnements en cas de client multiple. Une fois fait, la websocket se connecte au broker et s'abonne via le protocole MQTT au différent topics demandé par le client. Après-coup, la websocket ne sert plus que de passerelle entre le broker et le client pour les différentes données de capteurs. Les données envoyées par le broker ne sont pas modifiées par la socket, seule une date est ajoutée puisque les données transmises via l'abonnement n'en contiennent pas.

3.3.7 Front

Le rôle principal des deux composants vus précédemment est de connecter le broker et la base de données au client et donc au front. Le front a été développé en Vue.js, un framework permettant une architecture mono page facile à implémenter. Notre interface cliente comporte donc trois pages différentes :

- La page de **Login**, première page rencontrée par l'utilisateur qui permet de s'inscrire et de se connecter.
- La page d'**Abonnement** qui permet au client de choisir les topics auxquels il souhaite s'abonner ou non.
- La page d'**Accueil** qui affiche à l'utilisateur les données des différents topics.

Commençons donc par la page de Login. Elle comporte deux blocs, un pour la connexion et un pour l'inscription. En cas de connexion, l'utilisateur est redirigé vers la page d'accueil. En effet, nous supposons que lors d'une simple connexion, le client est là pour voir les différentes mesures prises par les capteurs. En revanche, en cas d'inscription, l'utilisateur est redirigé vers la page d'abonnement puis que ce dernier ne s'est encore abonnée à aucun topics.

Figure 14: Page login

Passons ensuite à la page d'abonnement. Cette dernière affiche à l'utilisateur une liste de topics préalablement envoyée par l'API Rest. L'affichage est dynamique, c'est-à-dire que si nous venons à ajouter plus de topics, la page n'aura aucun problème à afficher plus de topics. Les topics sont proposés sous la forme de boutons qui sont enfoncés ou non en fonction de la sélection du client. En cas d'arrivée sur la page pour un client ayant déjà des topics sélectionnés, ils apparaîtront présélectionnés. Après avoir sélectionné ses abonnements, l'utilisateur presse le bouton "OK" et est redirigé automatiquement vers la page d'accueil où les mesures des différents topics sont affichées.

Figure 15: Page abonnement - Topics non sélectionné

Figure 16: Page abonnement - Topics sélectionné

Pour finir, nous avons donc la page d'abonnement. Nous y retrouvons principalement les graphiques des différents topics. Nous avons utilisé la librairie Chart.js pour les générer. Nous

avons la possibilité de régler la période d’affichage des données avec un choix entre l’affichage des données de la dernière heure, des dernières six heures, des dernières douze heures, du dernier jour, de la semaine passé ou de l’entièreté des données collectées. De même que pour l’abonnement des topics sur la page d’abonnement, l’affichage des graphiques est dynamique et dépend du nombre d’abonnements choisi par le client. Sur la page d’accueil, nous retrouvons également des boutons pour aller à la page d’abonnement dans le cas où l’utilisateur voudrait les mettre à jour ainsi qu’un bouton de déconnexion.

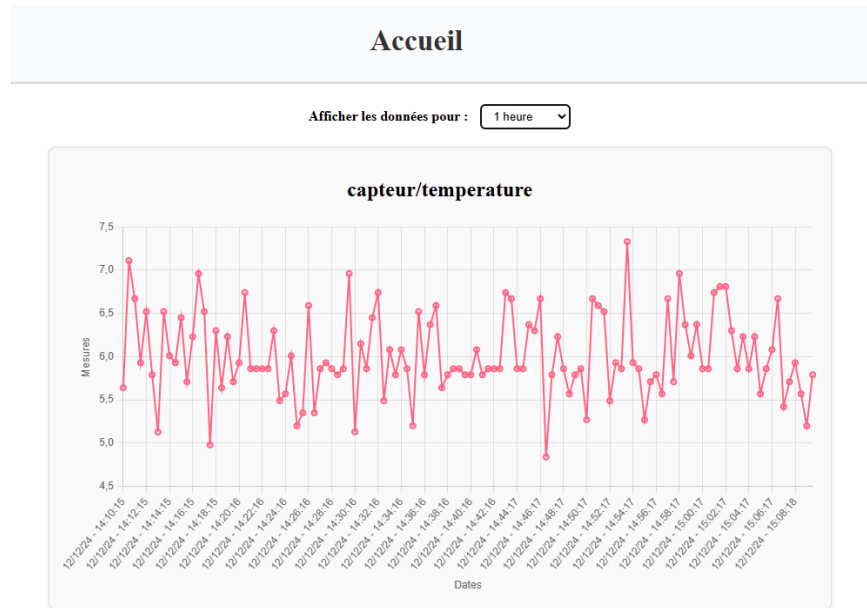


Figure 17: Page accueil

Pour conclure, notre front, développé en Vue.js, comporte trois pages distinctes permettant de *s’inscrire / se connecter*, de *s’abonner / se désabonner* aux différents topics et enfin une page d’affichage des données.

3.3.8 Orchestration Kubernetes

Afin d’assurer un déploiement cohérent et scalable, chaque composant développé précédemment a été conteneurisé à l’aide de Docker en utilisant des Dockerfiles distincts pour le frontend, l’API et le MQTT listener. De plus, des images préexistantes ont été utilisées pour le broker MQTT et la base de données MySQL, avec l’ajout de volumes persistants (PVC - Persistent Volume Claims) pour garantir la conservation des données même en cas d’arrêt du serveur.

L’orchestration a été réalisée avec K3S, une version allégée de Kubernetes, permettant de démarrer chaque composant de manière indépendante et de gérer automatiquement leur redémarrage en cas de panne. Cette orchestration a parfaitement fonctionné pour les services backend, notamment le broker MQTT, la base de données MySQL et le MQTT listener.

Cependant, des difficultés ont été rencontrées pour le déploiement des services frontend, API et WebSocket via K3S. Malgré ces limitations, nous avons réussi à valider le fonctionnement global de l’architecture en combinant l’orchestration K3S pour les composants backend avec une exécution locale des services API et frontend. Cette approche a permis de tester avec succès l’envoi et la

réception de messages : le frontend et l'API en local ont pu interagir avec les composants déployés dans l'architecture K3S pour la transmission et le stockage des données.

En conclusion, bien que l'intégration complète du frontend et de l'API dans l'orchestration K3S n'ait pas été réalisée, les résultats obtenus démontrent la viabilité de l'architecture et valident son bon fonctionnement dans un environnement hybride.

3.4 Conclusion

Ce chapitre a détaillé la mise en œuvre complète de l'architecture IoT, en décrivant les outils utilisés, les étapes d'implémentation des différents composants et les résultats obtenus. La conception repose sur une intégration cohérente entre les microcontrôleurs ESP32, le broker MQTT, la base de données MySQL, le serveur WebSocket, l'API REST et le frontend.

Une attention particulière a été portée à l'orchestration et la gestion des services grâce à Kubernetes. Cette approche a permis de déployer de manière robuste et scalable les services critiques, notamment le broker MQTT et la base de données MySQL, en s'appuyant sur des conteneurs Docker. Cette kubernetesisation assure une meilleure gestion des ressources, une haute disponibilité des services, ainsi qu'une scalabilité facilitée en cas de montée en charge.

Les tests menés ont validé le bon fonctionnement de l'architecture dans sa globalité, en assurant une communication fiable des données collectées par les capteurs, leur transmission via le broker MQTT, et leur stockage dans la base de données. La mise en place d'une WebSocket pour les échanges en temps réel, couplée à une API REST pour la gestion des données historiques et des utilisateurs, permet une interaction fluide et dynamique entre le client et le backend.

En conclusion, l'architecture implémentée répond aux exigences initiales en termes de performances, d'évolutivité et de fiabilité. L'utilisation de Kubernetes apporte une solution moderne et flexible pour gérer les composants critiques du système, tout en préparant l'architecture à d'éventuelles évolutions futures.

4 Conclusion

Ce projet avait pour objectif de concevoir et de déployer une architecture IoT basée sur le protocole MQTT capable de répondre aux exigences de scalabilité, de fiabilité et de performance dans un environnement connecté. En utilisant des ESP32, un broker MQTT, une base de données MySQL et une interface utilisateur moderne en Vue.js, nous avons mis en place un système modulaire et robuste permettant la collecte, la transmission et l’affichage des données en temps réel.

L’intégration de K3S, pour orchestrer les services backend a permis de valider la capacité de l’architecture à gérer efficacement les ressources et à garantir un déploiement automatisé des composants critiques. Bien que des difficultés aient été rencontrées pour le déploiement du frontend et de l’API dans cet environnement, une solution hybride a permis de valider le bon fonctionnement global de l’architecture, en assurant la communication entre les différents services. Toutefois, certaines améliorations restent à envisager :

- **Gestion d’actionneurs** : implémentation de boutons dans le frontend pour contrôler des actionneurs tels que l’allumage de lumières.
- **Compatibilité réseau** : rendre les adresses IP dynamiques afin de permettre une meilleure flexibilité lors du déploiement.
- **Respect total du Publish–subscribe pattern** : s’assurer que tous les composants suivent rigoureusement ce modèle pour optimiser les communications MQTT.
- **Déploiement complet** : finaliser la kubernetesisation du frontend et des backend API afin d’assurer un déploiement automatisé et intégré.
- **Augmentation de la sécurité** : sécuriser davantage les communications et les accès aux services en ajoutant des mécanismes d’authentification et de chiffrement (par exemple, TLS pour MQTT).

Malgré ces défis, nos tests ont validé le bon fonctionnement global de l’architecture. Nous avons réussi à établir une communication efficace entre les différents services pour l’envoi et la réception de messages, même dans un environnement hybride avec des composants déployés localement et sur K3S.

En conclusion, ce projet illustre l’intérêt et l’efficacité du protocole MQTT dans les architectures IoT modernes, tout en démontrant la valeur ajoutée des solutions d’orchestration comme Kubernetes. Avec les améliorations envisagées, notamment en matière de compatibilité réseau, d’actionneurs et de déploiement complet des services, cette architecture a le potentiel de devenir une solution scalable et performante pour les applications IoT à grande échelle.

Bibliographie

- [1] “DBeaver community | free universal database tool.” (), [Online]. Available: <https://dbeaver.io/> (visited on 12/10/2024).
- [2] “Eclipse mosquito,” Eclipse Mosquitto. (Jan. 8, 2018), [Online]. Available: <https://mosquitto.org/> (visited on 12/10/2024).
- [3] “Express - node.js web application framework.” (), [Online]. Available: <https://expressjs.com/> (visited on 12/10/2024).
- [4] E. T. Inc. “MQTTX: your all-in-one MQTT client toolbox,” mqttx.app. (), [Online]. Available: <https://mqttx.app/> (visited on 12/10/2024).
- [5] *IoT devices installed base worldwide 2015-2025*, en. [Online]. Available: <https://www-statista-com.passerelle.univ-rennes1.fr/statistics/471264/iot-number-of-connected-devices-worldwide/> (visited on 12/05/2024).
- [6] “K3s.” (), [Online]. Available: <https://k3s-io.github.io/> (visited on 12/10/2024).
- [7] “Learn JavaScript online - courses for beginners - javascript.com.” (), [Online]. Available: <https://www.javascript.com/> (visited on 12/10/2024).
- [8] *MQTT - The Standard for IoT Messaging*. [Online]. Available: <https://mqtt.org/> (visited on 12/07/2024).
- [9] “MySQL.” (), [Online]. Available: <https://www.mysql.com/> (visited on 12/10/2024).
- [10] “Node.js — run JavaScript everywhere.” (), [Online]. Available: <https://nodejs.org/en> (visited on 12/10/2024).
- [11] N. O’Leary, *Knolleary/pubsubclient*, original-date: 2009-02-02T11:07:58Z, Dec. 9, 2024. [Online]. Available: <https://github.com/knolleary/pubsubclient> (visited on 12/10/2024).
- [12] *Production-Grade Container Orchestration*, en. [Online]. Available: <https://kubernetes.io/> (visited on 12/07/2024).
- [13] “Software | arduino.” (), [Online]. Available: <https://www.arduino.cc/en/software> (visited on 12/10/2024).
- [14] “Visual studio code - code editing. redefined.” (), [Online]. Available: <https://code.visualstudio.com/> (visited on 12/10/2024).
- [15] “Vue.js.” (), [Online]. Available: <https://vuejs.org/> (visited on 12/10/2024).
- [16] “WebStorm: the JavaScript and TypeScript IDE, by JetBrains,” JetBrains. (), [Online]. Available: <https://www.jetbrains.com/webstorm/> (visited on 12/10/2024).